

API PART III

Materials Needed:

- A computer with Flutter installed.
- A code editor (like VS Code or Android Studio).
- The provider package installed in your Flutter project.

Instructions:

Use the previous code to before starting.
Replicate the code snippet.
Follow and implement the enhancement instructions.

Steps:

In your **surname_advmobprogAY2627 (root)** under it **surname_advmobprog** make the following

Clean the **pubspec.yaml (sample only)** – add **shared_preferences**

```
name: robles_advmobprog
description: "A new Flutter project."
publish_to: 'none'

version: 1.0.0+1

environment:
  sdk: ^3.10.1

dependencies:
  flutter:
    sdk: flutter

  cupertino_icons: ^1.0.8
  provider: ^6.1.5+1
  flutter_screenutil: ^5.9.3
  flutter_dotenv: ^6.0.1
  http: ^1.6.0
  shared_preferences: ^2.5.5

dev_dependencies:
  flutter_test:
    sdk: flutter

  flutter_lints: ^6.0.0

flutter:

  assets:
    - assets/
    - assets/icons/
    - assets/images/
  fonts:
    - family: Poppins
      fonts:
        - asset: assets/fonts/Poppins-Regular.ttf
  uses-material-design: true
```

Create these folders and files under the *project* folder with this **design pattern**.

```
robles_advmobprog/
├── .dart_tool/
├── android/
├── assets/
│   ├── fonts/
│   ├── icons/
│   ├── images/
│   └── .env
├── build/
├── ios/
├── lib/
│   ├── models/
│   │   ├── cart.dart
│   │   ├── product.dart
│   │   └── user.dart
│   ├── providers/
│   │   └── theme_provider.dart
│   ├── screens/
│   │   ├── cart_screen.dart
│   │   ├── detail_screen.dart
│   │   ├── home_screen.dart
│   │   ├── product_screen.dart
│   │   ├── profile_screen.dart
│   │   ├── settings_screen.dart
│   │   ├── signin_screen.dart
│   │   └── splash_screen.dart
│   ├── services/
│   │   ├── cart_service.dart
│   │   ├── product_service.dart
│   │   └── user_service.dart
│   ├── widgets/
│   │   └── custom_text.dart
│   ├── constants.dart
│   └── main.dart
├── test/
├── web/
├── .gitignore
├── .metadata
├── analysis_options.yaml
├── pubspec.lock
├── pubspec.yaml
└── README.md
```



Source Code

service/user_service.dart

```
import 'dart:convert';
import 'package:http/http.dart';
import 'package:shared_preferences/shared_preferences.dart';
import '../constants.dart';
import '../models/user.dart';

class UserService {
  Map<String, dynamic> data = {};

  Future<Map<String, dynamic>> loginUser(String username, String password) async {
    final response = await post(
      Uri.parse('$host/auth/login'),
      headers: {'Content-Type': 'application/json'},
      body: jsonEncode({
        'username': username,
        'password': password,
        'expiresInMins': 60,
      })),
    );

    if (response.statusCode == 200) {
      data = jsonDecode(response.body);
      await saveUserData(data);
      return data;
    } else {
      throw Exception(response.body);
    }
  }

  /// **Save User Data to SharedPreferences**
  /// Save user data from API response based on User model
  Future<void> saveUserData(Map<String, dynamic> userData) async {
    final prefs = await SharedPreferences.getInstance();
    final user = User.fromJson(userData);

    await prefs.setInt('id', user.id);
    await prefs.setString('username', user.username);
    await prefs.setString('email', user.email);
    await prefs.setString('firstName', user.firstName);
    await prefs.setString('lastName', user.lastName);
    await prefs.setString('gender', user.gender);
    await prefs.setString('image', user.image);
    await prefs.setString('accessToken', user.accessToken);
    await prefs.setString('refreshToken', user.refreshToken);

    // Support generic token key if present in API response
    if (userData.containsKey('token')) {
      await prefs.setString('token', userData['token'] ?? '');
    } else if (user.accessToken.isNotEmpty) {
      await prefs.setString('token', user.accessToken);
    }
  }

  /// Retrieve user data from SharedPreferences
  Future<Map<String, dynamic>> getUserData() async {
    final prefs = await SharedPreferences.getInstance();

    return {
      'id': prefs.getInt('id') ?? 0,
      'username': prefs.getString('username') ?? '',
      'email': prefs.getString('email') ?? '',
      'firstName': prefs.getString('firstName') ?? '',
      'lastName': prefs.getString('lastName') ?? '',
      'gender': prefs.getString('gender') ?? '',
      'image': prefs.getString('image') ?? '',
      'accessToken': prefs.getString('accessToken') ?? '',
      'refreshToken': prefs.getString('refreshToken') ?? '',
      'token': prefs.getString('token') ?? prefs.getString('accessToken') ?? '',
    };
  }

  /// Retrieve User model from SharedPreferences
  Future<User> getUser() async {
    final userData = await getUserData();
    return User.fromJson(userData);
  }

  /// **Check if User is Logged In**
  Future<bool> isLoggedIn() async {
    final prefs = await SharedPreferences.getInstance();
    final token = prefs.getString('accessToken') ?? prefs.getString('token');
    return token != null && token.isNotEmpty;
  }

  /// **Logout and Clear User Data**
  Future<void> logout() async {
    try {
      final prefs = await SharedPreferences.getInstance();
      await prefs.clear();
    } catch (e) {
      throw Exception('Failed to log out: $e');
    }
  }
}
```



screen/splash_screen.dart

```
Future<void> _checkAuthentication() async {
  await Future.delayed(const Duration(milliseconds: 1500));

  final loggedIn = await _userService.isLoggedIn();

  if (!mounted) return;

  if (loggedIn) {
    final userData = await _userService.getUserData();
    if (!mounted) return;
    Navigator.pushReplacementNamed(
      context,
      '/home',
      arguments: userData,
    );
  } else {
    Navigator.pushReplacementNamed(context, '/signin');
  }
}
```

screen/signin_screen.dart

```
void _login() async {
  UserService userService = UserService();
  setState(() {
    _isLoading = true;
  });
  if (_formKey.currentState!.validate()) {
    try {
      final response = await userService.loginUser(
        _usernameController.text,
        _passwordController.text,
      );

      // Save user data to SharedPreferences
      await userService.saveUserData(response);

      if (!mounted) return;
      setState(() {
        _isLoading = false;
      });

      Navigator.pushReplacementNamed(context, '/home', arguments: response);
    } catch (e) {
      if (!mounted) return;
      setState(() {
        _isLoading = false;
      });
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text('Login failed: ${e.toString()}')),
      );
    }
  }
}
```

Enhancements:

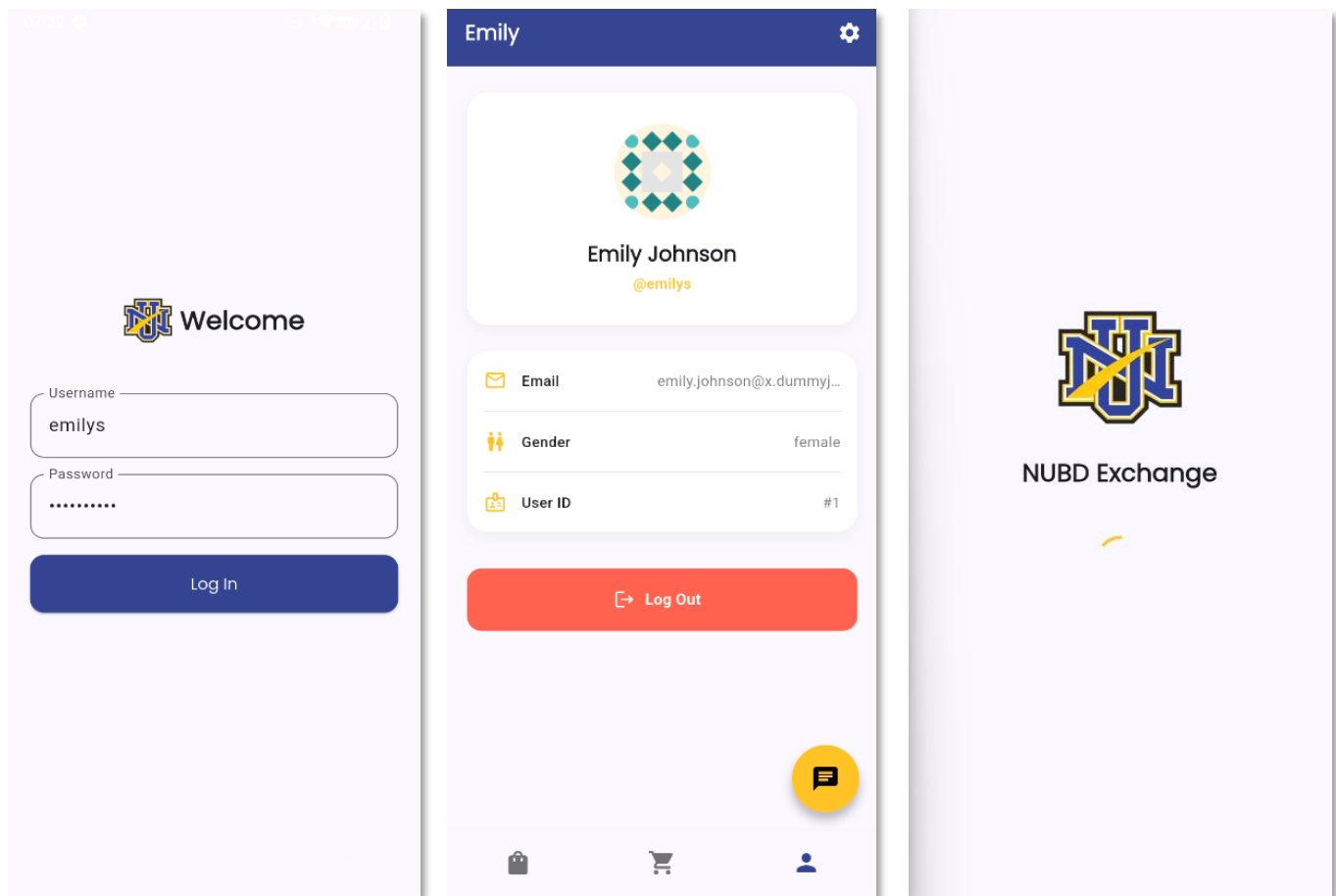
Comment on the source code each enhancement you've been implemented.

Enhancement 1: Make your own UI for the splash_screen implementing the persistent authentication.

Enhancement 2: Make your own UI for the sign_screen implementing the user_service and the authentication logic above.

Enhancement 3: Using the user_service create your own user.dart (model) implementing it on this project and rendering the data on the profile_screen creating UI on it. Based on the saved user data render the cart by userId

Sample Output:



Laboratory Discussion: Discuss how the user model, services and screen interact with each other to render the API endpoint going to the profile_screen. Discuss the updated design pattern in this activity. Also discuss how to use the saved data in rendering the cart_screen by user id.

Git Instruction:

Update first the README.md file adding the discussion part for example:

Lab Activity 4: discussion

after the hands-on and updating the README.md (must be on root folder surname-advmobprogAY2627)

1. git checkout -b lab_act4
2. git add .
3. git commit -m "lab_act4"
4. git push origin lab_act4

Activity Rubrics

Criteria	Excellent (4 pts)	Good (3 pts)	Fair (2 pts)	Poor (1 pt)	Points
Code Implementation	Code runs flawlessly, includes all required features (placeholders, avatars, widgets for like/comment).	Code mostly correct, minor errors, or missing one minor feature.	Noticeable errors or missing two or more features.	Code does not run or is significantly incomplete.	
Design Consistency	UI closely resembles intended layout, proper spacing, alignment, and styling.	Layout mostly accurate, minor spacing or alignment issues.	Layout deviates significantly from the intended design or appears incomplete.	Layout is unrecognizable or does not follow the instructions.	
Use of Widgets and Enhancements	Properly implements widgets for placeholders, avatars, and like/comment buttons.	Most widgets are correctly implemented, with minor issues or missing one enhancement.	Widgets are inconsistently used or missing two or more enhancements.	Widgets are not used, or enhancements are not implemented.	
Code Organization and Readability	Code is well-structured, follows best practices, and adheres to the specified file organization.	Code is mostly organized and readable, minor deviations from the required structure.	Code is poorly organized or does not follow the specified structure.	Code is disorganized and difficult to understand.	
Back-End Integration	Back-end integration works flawlessly with efficient API calls, secure data handling, and real-time updates; no errors or downtime.	Back-end integration is mostly correct with only minor API, data, or response handling issues.	Back-end integration has noticeable flaws or inefficiencies; multiple API calls fail or data handling is inconsistent.	Back-end integration has major issues; frequent failures, poor response handling, or insecure practices.	
Other Comments/ Observations:				Total Score	
				Rating: (Total Score/20) * 100	